ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Algebraic Data Types

With the development of the last few lectures, we have most of our language fundamentals in place: functions, primitive types, their reduction rules and a sound type system. Now it's time to look at compound types, to account for richer representations of data structures such as those available in practical programming languages.

We start with tuples, records and variants, conventionally referred to as **Algebraic Data Types (ADTs)**. The two fundamental mechanisms that support datatypes are **constructors** and **selectors**:

- a datatype **constructor** creates the values of the datatype by assembling multiple components;
- a datatype **selector** extracts the components assembled in the datatype.

As we will discuss shortly, *algebraic* refers to the constructors employed to assemble their components, which are qualified as *algebraic* as they are reminiscent of the product and sum operators found in algebra.

Datatype Constructors

Constructors are the fundamental mechanism to build datatypes: they are best understood as functions that *inject* values from one or more domain (type) into the domain of the datatype being defined.

Tuples, records and variants are all created by corresponding constructors. We look at each of them below.

Tuples

Tuples are the familiar structures we are used to see in mathematics. In formal systems as well as in programming languages, they are introduced as **structural types**: for each n, the n-ary tuple constructor $(\cdot)_n : T_1 \times \cdots \times T_n \to (T_1, \ldots, T_n)$ collects n components of the respective types and assembles them into a compound value. Thus we have:

$(\mathsf{true}, \mathsf{false}) : (\mathsf{Bool}, \mathsf{Bool})$

$(\text{"aaa"}, \mathsf{true}) : (\mathsf{String}, \mathsf{Bool})$

$(1, 5, 2, 10) : (\mathsf{Int}, \mathsf{Int}, \mathsf{Int}, \mathsf{Int})$

$(3, \mathsf{true}, 2.3, \text{"aaa"}) : (\mathsf{Int}, \mathsf{Bool}, \mathsf{Double}, \mathsf{String})$

$(\text{"bb"}, \lambda(\text{x} : \text{Bool}).\text{x} \,||\, \text{true}) : (\text{String}, \text{Bool} \to \text{Bool})$

Since we need a tuple constructor for every n and every combination of component types, we must assume that the constructor is either type-indexed (one for each combination) or polymorphic. In either case, the typing relation is captured by the following rule:

$$\frac{\Gamma \vdash M_1 : T_1 \quad \cdots \quad \Gamma \vdash M_n : T_n}{\Gamma \vdash (M_1, \ldots, M_n) : (T_1, \ldots, T_n)} \ (\text{T-Tuple})$$

Alternatively, we may take n-tuples as built by iterated pairing. For instance, the triple type (T_1, T_2, T_3) may be constructed as $((T_1, T_2), T_3)$ or as $(T_1, (T_2, T_3))$: these types are different, and the nesting affects the way selectors access the components.

Example. Let us derive the type of the tuple $(3, \text{true})$ in the empty context. We need to show $\vdash (3, \text{true}) : (\text{Int}, \text{Bool})$:

$$\frac{\vdash 3 : \text{Int} \qquad \vdash \text{true} : \text{Bool}}{\vdash (3, \text{true}) : (\text{Int}, \text{Bool})} \ (\text{T-Tuple})$$

Records

Records are a generalization of tuples in which each component (field) is associated with a label (in programming languages, simply an identifier). Given n distinct labels $\ell_1, \ldots, \ell_n$, a record constructor collects n components and associates each with the corresponding label. The typing rule has the following format:

$$\frac{\Gamma \vdash M_1 : T_1 \quad \cdots \quad \Gamma \vdash M_n : T_n}{\Gamma \vdash \{\ell_1 = M_1, \ldots, \ell_n = M_n\} : \{\ell_1 : T_1, \ldots, \ell_n : T_n\}} \ (\text{T-Rec})$$

Examples

$\{\text{fst} = 5, \text{snd} = 3\} : \{\text{fst} : \text{Int},\ \text{snd} : \text{Int}\}$

$\{\text{city} = \text{"Boston"}, \text{street} = \text{"Worcester"}, \text{number} = 15\} : \{\text{city} : \text{String},\ \text{street} : \text{String},\ \text{number} : \text{Int}\}$

$\{\text{name} = \text{"Alice"},\ \text{age} = 30,\ \text{active} = \text{true}\} : \{\text{name} : \text{String},\ \text{age} : \text{Int},\ \text{active} : \text{Bool}\}$

Since components are identified by their labels and the labels are all distinct, their order is irrelevant: records and record types that differ only in the ordering of their fields are considered equal. Thus

$\{\text{fst} = \text{"a"}, \text{snd} = 3\} \ \cong \{\text{snd} = 3, \text{fst} = \text{"a"}\}$ and similarly

$\{\text{fst} : \text{String}, \text{snd} : \text{Int}\} \ \cong \{\text{snd} : \text{Int}, \text{fst} = \text{String}\}$.

This is a key difference from tuples, where position determines identity: $(\text{"a"}, 3)$ and $(3, \text{"a"})$ are different and have distinct types.

Like tuples, Records may contain components of any type, including function types. For example:

$$\{\mathsf{fun} = \lambda(\mathsf{x} : \mathsf{Int}).\ \mathsf{x} + 1,\ \mathsf{val} = 5\} : \{\mathsf{apply} : \mathsf{Int} \to \mathsf{Int},\ \mathsf{val} : \mathsf{Int}\}$$

Variants

Tuples and records assemble multiple pieces of data forming a compound structure: a pair contains both a first and a second component, a record contains all of its fields. These are *product typesé: their values are formed by combining one component from each factor, just as elements of a Cartesian product are tuples of elements drawn from each set.

But in many situations, a value may take one of several alternative forms:

- a shape might be a circle or a rectangle, not both at once
- computation might return a result or signal an error.
- a node in a data structure might be a leaf or an internal node with children.

What we need is a type that describes a choice among alternatives within a single domain, where each alternative may carry different data. These are *sum types*: their values belong to one of several disjoint alternatives, just as elements of a disjoint union belong to exactly one of the constituent sets.

The foundational mechanisms that support this kind of programming are *variants* and *variant types*. A variant type is defined by a collection of **data constructors**, each of which specifies one of the possible alternatives.

Unlike the tuple and record constructors, which assemble all their components into a single compound value, a data constructor for a variant *injects* a value into one particular alternative of the type.

Each constructor has a name and specifies the types of the arguments it expects. To illustrate, we may define *shapes* with the following variant type

$$\langle\ \mathsf{Circ}(\mathsf{Double}) \mid \mathsf{Square}(\mathsf{Double}) \mid \mathsf{Rect}(\mathsf{Double}, \mathsf{Double})\ \rangle$$

The three data constructors Circ, Square and Rect serve a dual purpose: at construction time, they tag the value so that we know which alternative was chosen; at selection time (via pattern matching, see below), they allow us to determine which alternative we are looking at and extract the components accordingly.

A data constructor with no arguments is called a *constant* (or *nullary*) constructor: it represents an alternative that carries no data. For example, in the *optional boolean* type below, the constructor None

signals the absence of a value, while the other alternative carries the value b (which is injected in the variant by the constuctore Some).

$$\langle\ \mathsf{None} \mid \mathsf{Some}(\mathsf{Bool})\ \rangle$$

When *all* constructors in a variant are constant, the type is an **enumeration** — a finite set of named values with no internal structure. For example, the type

$$\langle\ \mathsf{Red} \mid \mathsf{Green} \mid \mathsf{Blue}\ \rangle$$

has three constant constructors and is nothing more than a three-element enumeration. The the booleans themselves can be seen as the enumeration $\mathsf{Bool} = \langle\ \mathsf{True} \mid \mathsf{False}\ \rangle$.

Typing variants: the problem

With tuples and records, the structure of a value fully determines its type:

- seeing $(3, \mathsf{true})$ is enough to conclude that the type is $(\mathsf{Int}, \mathsf{Bool})$,
- seeing $\{\mathsf{fst} = 5, \mathsf{snd} = 3\}$ is enough to conclude that the type is $\{\mathsf{fst} : \mathsf{Int}, \mathsf{snd} : \mathsf{Int}\}$.

The syntactic form of the constructor, together with the types of its components, leaves no ambiguity.

With variants, the situation is different. A variant value like $\mathsf{Circ}(3.0)$ shows us only *one* of the alternatives: we can see that the constructor is Circ and the argument has type Double, but we cannot tell from this alone which variant the value belongs to. The constructor Circ could be part of any number of different variant types:

$$\begin{aligned}
\mathsf{Circ}(3.0) &: \langle\ \mathsf{Circ}(\mathsf{Double})\ \rangle \\
\mathsf{Circ}(3.0) &: \langle\ \mathsf{Circ}(\mathsf{Double}) \mid \mathsf{Rect}(\mathsf{Double}, \mathsf{Double})\ \rangle \\
\mathsf{Circ}(3.0) &: \langle\ \mathsf{Circ}(\mathsf{Double}) \mid \mathsf{Square}(\mathsf{Double})\ \rangle \\
\mathsf{Circ}(3.0) &: \ldots
\end{aligned}$$

As we see, the type of a variant value is not determined by its structure alone.

Typing variants: the solution

To solve the problem, we resort to richer typing judgements, which we extend with contextual information about the variant types that have been defined.

For that purpose, we introduce a **constructor environment** Δ that keeps the variant types in scope and maps each constructor name to its *parent type* — the variant type it belongs to.

This is the environment the type checker consults when it encounters a constructor application like $\mathsf{Circ}(3.0)$: it looks up Circ in Δ, finds that it belongs to the variant type describing shapes, and retrieves the

expected argument type from the defining equation.

The critical requirement is **constructor uniqueness**: no constructor name may appear in two different variants. If this condition is met, seeing $\mathsf{Circ}(3.0)$ suffices to determine that the value belongs to the variant describing shapes: the constructor name, together with Δ, uniquely resolves the type.

With Δ in place, the typing judgement becomes $\Delta;\, \Gamma \vdash M : T$, and the typing rule for variant construction is:

$$\frac{\Delta(\mathsf{C}_j) = \langle \ldots \mid \mathsf{C}_j(T_j) \mid \ldots \rangle \quad \Delta;\, \Gamma \vdash M : T_j}{\Delta;\, \Gamma \vdash \mathsf{C}_j(M) : \Delta(\mathsf{C}_j)} \quad (\text{T-Variant})$$

The rule reads: the type checker looks up the constructor C_j in Δ to find its parent type and extract the type T_j expected by C_j. If the argument M has type T_j, then $\mathsf{C}_j(M)$ has the type $\Delta(\mathsf{C}_j)$.

We tacitly assume that Δ is *well formed*, i.e. that constructor names occur uniquely in Δ.

Recursive Types

The datatypes introduced so far — tuples, records, and variants — are all *finite* in structure: once we fix the number and types of their components, the resulting values have a bounded depth. This is sufficient for many purposes, but some of the most common data structures in programming — lists, trees, streams — are inherently *unbounded*: a list, for instance, can be of any length, and a tree can have any depth.

To see the problem, consider defininining **integer lists** with a variant type, according to the following intuition:

> A int-list is either empty, or a *cons cell* built around an integer and a int-list.

Trying a variant type, we have:

$$\langle \mathsf{Empty} \mid \mathsf{Cons}(\mathsf{Int}, ???) \rangle$$

The problem is that to fill the second argumento for Cons we would need to refer to the representing int-list. But int-list is precisely the type we are trying to define!

To describe such structures, we need types that refer to *themselves* in their own definition, i.e. recursive types.

The Rec operator: a type-level fixpoint

To handle recursive types cleanly, we introduce a **type-level recursion operator** Rec that makes the fixed-point structure explicit. We extend the syntax of type expressions with:

$$\mathsf{Rec\ X.}\ T$$

where X is a *type variable* and T is a type expression in which X may occur free.

This provide us with a new tool to define integer lists, as we may use a recursive type and the recursion variable to refer to the type being defined.

$$\mathsf{Rec\,X.\langle Empty \mid Cons(Int, X)\rangle}$$

The key property of the recursione operator is the **unfolding principle**: the type Rec X. T is equal to (or, depending on the formulation, *isomorphic to*) the type obtained by substituting Rec X. T for every free occurrence of X in T:

$$\mathsf{Rec\ X.}\ T\ = [\mathsf{Rec\ X.}\ T\ /\ \mathsf{X}]\,T$$

Back to our intlists, introducing the shorthand $\mathsf{IntList} \equiv \mathsf{Rec\,X.\langle Empty \mid Cons(Int, X)\rangle}$, by the unfolding principle we have that:

$$\mathsf{IntList} = \langle\ \mathsf{Empty} \mid \mathsf{Cons(Int,\ IntList})\ \rangle$$

Now we see clearly how to construct values of the type, i.e. by iterated applications of the constructors:

> Empty — the empty list
>
> $\mathsf{Cons}(5, \mathsf{Empty})$ — the singleton list $[5]$
>
> $\mathsf{Cons}(5, \mathsf{Cons}(3, \mathsf{Cons}(4, \mathsf{Empty})))$ — the list $[5, 3, 4]$

Recursive Types vs Recursive Equations The above equation for IntList is the familiar equation adopted in most programming languages.

This is a convenient and readable notation, but one should be aware that there is a fundamental difference between

- *deriving equations* as a conseguence of the unfolding princilple associated with the recursion operator Rec, as we have done in our foundational construction, and
- *introducing equations as a definition tool* as it is done in programming language.

In programming languages, instead, type presentations are *nominal*, i.e. they require type names to be able to make sense of recursively defined types via recursive equations.

With the recursion operator, we may keep the presentation purely *structural*, as types are entily described by their components. Names, and defining erquations may still be useful in structural presentations in case we wish to use convenient shorthands, but they are pure syntactic sugar.

Examples

Integer binary trees. Similarly to lists, the type of binary trees with integer leaves is:

$$\mathsf{IntTree} \equiv \mathsf{Rec\ X.}\ \langle\, \mathsf{Leaf}(\mathsf{Int}) \mid \mathsf{Node}(\mathsf{X}, \mathsf{X}) \,\rangle$$

By the unfolding principle: $\mathsf{IntTree} = \langle\, \mathsf{Leaf}(\mathsf{Int}) \mid \mathsf{Node}(\mathsf{IntTree}, \mathsf{IntTree}) \,\rangle$. Some values:

$\mathsf{Leaf}(42)$ — a single leaf

$\mathsf{Node}(\mathsf{Leaf}(1), \mathsf{Leaf}(2))$ — a tree with one internal node and two leaves

$\mathsf{Node}(\mathsf{Leaf}(1), \mathsf{Node}(\mathsf{Leaf}(2), \mathsf{Leaf}(3)))$ — a tree with three leaves

Streams. The Rec operator is not limited to variants. Any type expression may serve as the body of a Rec. To illustrate, consider the following recursive *pair* type:

$$\mathsf{IntStream} \equiv \mathsf{Rec\ X.}\ (\mathsf{Int}, \mathsf{X})$$

By the unfolding principle, $\mathsf{IntStream} = (\mathsf{Int}, \mathsf{IntStream})$. A value of this type is a pair of an integer (the current element) and another stream (the rest).

Note that, being there no base case, streams are inherently infinite. Such structures are perfectly legitimate values in a lazy language, but constructing *finite* values becomes impossible. In the context of strict (eager) evaluation — which is what we have been assuming throughout — a recursive type without a base case would not admit any (finite) value: any attempt to build one would require building another, leading to non-termination.

Self-application. The type

$$\mathsf{U} \equiv \mathsf{Rec\ X.}\ \mathsf{X} \to \mathsf{X}$$

is the type of functions that can be applied to themselves. In fact, it is sometimes referred to as the *universal* type for the λ calculus, as all λ terms may be given this type. To see that, note that by the unfolding principle, we have:

$$\mathsf{U} = \mathsf{U} \to \mathsf{U}$$

Interestingly, by virtue of this equation, we may also type self applications such as $(\lambda x.x\ x)$. In the same way, As a result, this kind of recursive type is what makes it possible to type the Ω combinator $(\lambda x.x\ x)(\lambda x.x\ x)$ and, more usefully, to type fixed-point combinators. We will not pursue this direction further here, but it illustrates the generality of the Rec mechanism.

Rec vs fix

The Rec operator is the type level counterpart of the fix operator we discussed for the typed λ-calculus.

The term fix. M is meant to capture the behaviour of recursive function definitions, based on the reduction fix. $M \to M\,(\text{fix. } M)$. In this construction, M is intended to be a function - as required for the typing rule (T-Fix) - otherwise the reduct $M\,(\text{fix. } M)$ would be a meaningless (stuck) term.

Then, if we take the special case when M is a λ-abstraction, we see that in two reductions we have:

$$\text{fix. } (\lambda \text{x. } M) \to (\lambda \text{x. } M)\,(\text{fix. } (\lambda \text{x. } M)) \to [\text{fix. } (\lambda \text{x. } M) \,/\, \text{x}]\, M$$

Now let's introduce the following shorhand notation $\text{FIX x.M} \equiv \text{fix. } (\lambda \text{x. M})$. Then, the reductions above read as

$$\text{FIX x. } M \to (\lambda \text{x. } M)\,(\text{FIX x. } M) \to [\text{FIX x.} M \,/\, \text{x}]\, M$$

In other words, $\text{FIX x. } M \to^* [\text{Fix x. } M \,/\, \text{x}]\, M$, which is the exact counterpart of the unfolding isomorphism provided by Rec over types.

The parallel between the two constructions can be explained more accurately reasoning in the opposite direction, by construing the type Rec X. T as the fixed point of the type operator λX. T, and Rec X. T as a shorthand notation for Rec.$(\lambda \text{X. } T)$. Summarizing:

	Term level	**Type level**
Operator	fix.λx. M	Rec.λX. T
Shorthand	FIX x. M	Rec X. T
Unfolds to	$[\text{FIX x.} M \,/\, \text{x}]\, M$	$[\text{Rec X. } T \,/\, \text{X}]\, T$
Meaning	fixed point of λx. M	fixed point of λX. T

Pattern Matching — A uniform selection mechanism

We've already seen pattern-matching at work in our reference programming languages. Now, we present it for what it really is: a fully general mechanism for selecting datatype components.

Fundamentally, pattern-matching is a mechanism for taking apart a value by *finding out which constructor* it was built with and *which components the constructor was applied to*. Based on that, it provides a uniform (in some cases the *only*) way to select the constituents of a compound datatype.

Patterns

Given a datatype constructor, a *pattern* is essentially a *semi-constructed* term, in which some of the component positions are left unspecified and represented by variables. We write

$$P[x_1, \ldots, x_n]$$

for a pattern P whose unspecified positions are the variables $x_1, \ldots, x_n$: the pattern represents all terms $P[M_1, \ldots, M_n]$ obtained by substituting the pattern variables with proper terms.

A term M *matches* a pattern $P[x_1, \ldots, x_n]$ if there exist terms $N_1, \ldots, N_n$ such that $M \equiv P[N_1, \ldots, N_n]$.

We distinguish two main kinds of patterns.

Variable Patterns. A variable x is a pattern that matches all terms. The anonymous variable (the *wildcard*) "_" also matches everything, but without creating a binding — a distinction that matters in matching expressions, as we will see shortly.

Constructor Patterns. Given any constructor, any semi-constructed term built from the constructor is a pattern. Examples:

- $(\mathsf{false}, \mathsf{x}, 1, _, \mathsf{z})$ is a 5-tuple pattern, matched by any 5-tuple that has false and 1 in the first and third position respectively. Notice that wildcards and variables may coexist within a pattern.
- $(\mathsf{false}, \mathsf{x}, 1, (\mathsf{y}, 3))$ is a nested 4-tuple pattern, matched by any quadruple whose first, third and fourth elements are false, 1 and a pair whose second element is 3 respectively.
- $\{\mathsf{fst} = 5, \mathsf{snd} = \mathsf{x}\}$ is a record pattern, matched by all pairs whose fst field is 5.
- $\mathsf{Cons}(5, \mathsf{Cons}(\mathsf{x}, \mathsf{Cons}(3, \mathsf{xs})))$ is an $\mathsf{IntList}$ pattern matched by all lists with at least three elements whose first and third elements are 5 and 3.

Further examples of matching. Let us trace which terms match the following patterns and what bindings result:

- $(\mathsf{x}, \mathsf{true})$ matches all pairs with second element true
- $\mathsf{Cons}(\mathsf{x}, \mathsf{Empty})$ matches all lists with exactly one element
- $\mathsf{Node}(\mathsf{Leaf}(\mathsf{x}), \mathsf{Leaf}(\mathsf{y}))$ matches all binary trees with one node
- $\{\mathsf{name} = \mathsf{x}, \mathsf{age} = _\}$ matches any record with fields name and age, binding only the name. The wildcard discards the age.

The following table further illustrates this behavior.

Pattern	Term	Matching result
$(\mathsf{x}, \mathsf{true})$	$(42, \mathsf{true})$	matches with $\mathsf{x} \mapsto 42$

Pattern	Term	Matching result
	$(42, \mathsf{false})$	does *not* match
	$(\text{"hi"}, \mathsf{true})$	matches with $\mathsf{x} \mapsto \text{"hi"}$
$\mathsf{Cons}(\mathsf{x}, \mathsf{Empty})$	$\mathsf{Cons}(7, \mathsf{Empty})$	matches with $\mathsf{x} \mapsto 7$
	$\mathsf{Cons}(7, \mathsf{Cons}(3, \mathsf{Empty}))$	does *not* match (tail is not Empty).
$\mathsf{Node}(\mathsf{Leaf}(\mathsf{x}), \mathsf{Leaf}(\mathsf{y}))$	$\mathsf{Node}(\mathsf{Leaf}(1), \mathsf{Leaf}(2))$	matches with $\mathsf{x} \mapsto 1, \mathsf{y} \mapsto 2$
	$\mathsf{Node}(\mathsf{Leaf}(1), \mathsf{Node}(\mathsf{Leaf}(2), \mathsf{Leaf}(3)))$	does not match

Existing programming languages provide further pattern forms, which we will discuss in the next lecture. Here, we move on to complete the discussion on pattern matching by introducing matching expressions.

Matching expressions

The formal device for putting patterns and composite datatypes to work is a *matching expression*. Different programming languages employ different syntax and naming, but the fundamental mechanism is the same: a term M is matched against a sequence of patterns, and when a match is found, a new binding is created for each pattern variable from the corresponding sub-term in M.

The general form of a matching expression is:

$$M \text{ match } \{P_1[\bar{x}_1] \Rightarrow M_1 \mid \cdots \mid P_n[\bar{x}_n] \Rightarrow M_n\}$$

The variables occurring in each pattern may occur free in the corresponding term M_i to the right of the arrow. When the expression is evaluated:

- the term M is matched against each of the patterns $P_1, P_2, \ldots P_n$
- if a matching pattern P_i is found, then $M \equiv P_i[\bar{N}_i]$ for suitable terms $\bar{N}_i$
- the evaluation proceeds with M_i, where each pattern variable is substituted by the corresponding sub-term found in M.

The following reduction rule formalizes this behavior. For readability, we present the simpler case of single-variable patterns; the generalization to multiple variables is straightforward.

$$P_i[N] \text{ match } \{(P_1[x_1] \Rightarrow M_1 \mid \cdots \mid P_n[x_n] \Rightarrow M_n)\} \;\rightarrow\; [N/x_i]M_i$$

If no pattern matches, the expression is stuck. Notice that in principle the term could match more than one pattern, which would break the Church-Rosser property. To solve the problem:

- **Foundational calculi** require that patterns be **disjoint**, so that the term matches at most one pattern.
- **Programming languages** take a more pragmatic approach: (i) **patterns are tried sequentially**, in the order they occur, and (ii) only the first matching clause is selected. If no pattern matches, the expression returns an error — a situation ruled out in practice by *exhaustiveness checking*, which verifies statically that the patterns cover all values of the scrutinized type.

Example: reducing a matching expression. Using the Shape variant, consider the following expression that computes the area of a shape:

$$\mathsf{Circ}(3.0)\ \ \mathsf{match}\ \{\mathsf{Circ}(\mathsf{r}) \Rightarrow \pi * \mathsf{r}^2 \mid \mathsf{Rect}(\mathsf{b}, \mathsf{h}) \Rightarrow \mathsf{b} * \mathsf{h}\}$$

- The value $\mathsf{Circ}(3.0)$ matches the first pattern $\mathsf{Circ}(\mathsf{r})$ with $\mathsf{r} \mapsto 3.0$. Instead, in

$$\mathsf{Rect}(5.0, 2.0)\ \ \mathsf{match}\ \{\mathsf{Circ}(\mathsf{r}) \Rightarrow \pi * \mathsf{r}^2 \mid \mathsf{Rect}(\mathsf{b}, \mathsf{h}) \Rightarrow \mathsf{b} * \mathsf{h}\}$$

- the value $\mathsf{Rect}(5.0, 2.0)$, does *not* match $\mathsf{Circ}(\mathsf{r})$, so the second pattern $\mathsf{Rect}(\mathsf{b}, \mathsf{h})$ is tried, matching with $\mathsf{b} \mapsto 5.0, \mathsf{h} \mapsto 2.0$.

Example: matching on a recursive type. The length of an integer list can be computed as:

$$\mathsf{Cons}(5, \mathsf{Cons}(3, \mathsf{Empty}))\ \mathsf{match}\ \{\mathsf{Empty} \Rightarrow 0 \mid \mathsf{Cons}(\mathsf{a}, \mathsf{rest}) \Rightarrow 1 + (\mathsf{rest}\ \mathsf{match}\ \ldots)\}$$

- The first pattern Empty does not match;
- The second pattern $\mathsf{Cons}(\mathsf{a}, \mathsf{rest})$ matches with $\mathsf{a} \mapsto 5, \mathsf{rest} \mapsto \mathsf{Cons}(3, \mathsf{Empty})$.

The expression then reduces to $1 + (\mathsf{Cons}(3, \mathsf{Empty})\ \mathsf{match}\ \ldots)$, and the process repeats recursively.

Typing matching expressions

The typing rule for matching expressions ensures type preservation. In the interest of readability, we give the rule for single-variable patterns.

$$\frac{\Delta;\, \Gamma \vdash M : S \qquad (\Delta;\, \Gamma, x_i : T_i \vdash P_i : S \quad \Delta;\, \Gamma, x_i : T_i \vdash M_i : T)_{i \in 1..n}}{\Delta;\, \Gamma \vdash M\ \mathsf{match}\ (P_1[x_1] \Rightarrow M_1 \mid \cdots \mid P_n[x_n] \Rightarrow M_n) : T} \quad (\text{T-Match})$$

The idea is that the scrutinized term M has some type S, each pattern P_i is compatible with S (binding each pattern variable to a value of the appropriate type), and all the branches M_i produce the same result type T

Example: typing the area expression. We verify that the following judgement is derivable.

$$\Delta; \Gamma \vdash \mathsf{Circ}(3.0)\ \ \mathsf{match}\ \{\mathsf{Circ}(\mathsf{r}) \Rightarrow \pi * \mathsf{r}^2 \mid \mathsf{Rect}(\mathsf{b}, \mathsf{h}) \Rightarrow \mathsf{b} * \mathsf{h}\} : \mathsf{Double}$$

We assume Δ contains the type definition $\mathsf{Shape} = \langle \mathsf{Circ}(\mathsf{Double}) \mid \mathsf{Rect}(\mathsf{Double}, \mathsf{Double}) \rangle$.

We verify that the area matching expression is well-typed by constructing the typing derivation.

- $\Delta; \Gamma \vdash \mathsf{Circ}(3.0) : \mathsf{Shape}$, which derives by (T-Variant), since $\Delta(\mathsf{Circ}) = \mathsf{Shape}$.
- For the first branch: $\Delta; \Gamma, \mathsf{r} : \mathsf{Double} \vdash \mathsf{Circ}(\mathsf{r}) : \mathsf{Shape}$ and $\Delta; \Gamma, \mathsf{r} : \mathsf{Double} \vdash \pi * \mathsf{r}^2 : \mathsf{Double}$
- For the second branch: $\Delta; \Gamma, \mathsf{b} : \mathsf{Double}, \mathsf{h} : \mathsf{Double} \vdash \mathsf{Rect}(\mathsf{b}, \mathsf{h}) : \mathsf{Shape}$ and $\Delta; \Gamma, \mathsf{b} : \mathsf{Double}, \mathsf{h} : \mathsf{Double} \vdash \mathsf{b} * \mathsf{h} : \mathsf{Double}$.

Both branches produce type Double, so the whole expression has type Double by (T-Match).

Exercises

1. Reduction and typing for datatype-specific selectors. Assume we extend our foundational presentation of datatypes with syntactic forms to express specific operators for selecting the components of tuples and records.

$$M \ ::= \ \cdots \mid M._j \mid M.\ell \mid \ \cdots$$

The intuition is as follow:

- $M._j$ selects the j-th component of the tuple M
- $M.\ell$ selects the component associated with the label ℓ in the record M.

Define the reduction and typing rules to capture the intended semantics and typing relation for the new syntactic forms.

2. Typing derivations. Construct full typing derivations (using Δ and Γ) for each of the following terms, assuming the definitions given in the lecture.

- $\mathsf{Cons}(1, \mathsf{Cons}(2, \mathsf{Empty}))$
- $\mathsf{Node}(\mathsf{Leaf}(42), \mathsf{Leaf}(0))$
- $\{\mathsf{fst} = \mathsf{Circ}(1.0),\ \mathsf{snd} = \mathsf{true}\}$

3. Unfolding recursive types. Write out the result of unfolding each of the following recursive type definitions once, and give two example values of each type.

- $\mathsf{NatList} = \mathsf{Rec\ X.}\ \langle\, \mathsf{Nil} \mid \mathsf{Cons}(\mathsf{Nat}, \mathsf{X}) \,\rangle$
- $\mathsf{StringTree} = \mathsf{Rec\ X.}\ \langle\, \mathsf{Leaf}(\mathsf{String}) \mid \mathsf{Branch}(\mathsf{X}, \mathsf{X}) \,\rangle$
- $\mathsf{NestedPair} = \mathsf{Rec\ X.}\ (\mathsf{Int}, \langle\, \mathsf{Stop} \mid \mathsf{More}(\mathsf{X}) \,\rangle)$

4. Writing recursive type definitions. For each of the following informal descriptions, write a recursive type definition using the $\mathsf{Rec\ X.\ T}$ notation.

- A type of *stacks* of booleans, where a stack is either empty or has a boolean on top of another stack.

- A type of *labelled binary trees*, where each internal node carries a String label and has two sub-trees, and each leaf carries an Int.
- A type of *non-empty lists* of integers: a non-empty list is either a single integer, or an integer followed by another non-empty list.

5. Option types. Consider the following type for optional values

$$\mathsf{OptionNat} = \langle \mathsf{NaN} \mid \mathsf{Some}(\mathsf{Nat}) \rangle$$

- Write a matching expression that computes the successor of an optional natural number: it should return $\mathsf{Some}(\mathsf{n}+1)$ when given $\mathsf{Some}(\mathsf{n})$, and NaN when given NaN.
- Write a matching expression that, given two values of type $\mathsf{OptionNat}$, returns their sum if both are present, and NaN otherwise.

6. Finite maps. Consider the following type representing finite mappings from natural numbers to natural numbers:

$$\mathsf{FiniteMap} = \mathsf{NaN} \to \mathsf{OptionNat}$$

The domain of finite maps is the set of inputs for which the result is $\mathsf{Some}(\mathsf{n})$. Define the following functions, in Haskell and / or Scala.

- $\mathsf{emptyMap} : \mathsf{FiniteMap}$
  the map with empty domain (i.e. the function that returns NaN for all inputs).
- $\mathsf{extendMap} : \mathsf{FiniteMap} \to \mathsf{Nat} \to \mathsf{Nat} \to \mathsf{FiniteMap}$
  the pam that applied to M k v returns a new map that behaves like M on all inputs except k, for which it returns $\mathsf{Some}(\mathsf{v})$.
- $\mathsf{lookup} : \mathsf{FiniteMap} \to \mathsf{Nat} \to \mathsf{OptionNat}$
  the map that returns the value associated with a key, or NaN if the key is not in the domain of the map.

7. Binary trees. Consider the type $\mathsf{IntTree} = \mathsf{Rec}\ \mathsf{X}.\ \langle\ \mathsf{Leaf}(\mathsf{Int}) \mid \mathsf{Node}(\mathsf{X},\mathsf{X})\ \rangle$.

- Write a matching expression that computes the sum of all the leaf values in an $\mathsf{IntTree}$.
- Write a matching expression that computes the depth of an $\mathsf{IntTree}$ (the depth of a Leaf is 0, the depth of a Node is 1 plus the maximum depth of its sub-trees).
- Write a matching expression that mirrors an $\mathsf{IntTree}$, i.e. swaps the left and right sub-trees at every internal node.

8. Nested patterns and wildcards. Consider the type $\mathsf{IntList} = \mathsf{Rec}\ \mathsf{X}.\ \langle \mathsf{Empty} \mid \mathsf{Cons}(\mathsf{Int},\mathsf{X}) \rangle$.

- Write a pattern that matches lists with *exactly* two elements.
- Write a pattern that matches lists with *at least* two elements, binding only the second element to a variable.

- Using pattern matching, write an expression that returns `true` if the first two elements of a list are equal, and `false` otherwise (including the case of lists with fewer than two elements). *Hint:* you may need more than two clauses.

CREDITS

The discussion on algebraic datatypes is adapted from Chapter 11 of Prof. Benjamin's Pierce's book on Types and Programming Languages. MIT Press. 2001. The treatment of recursive types draws on Chapter 20 of the same book.